

ASSIGNMENT - 2

CSC-303 – Computer Networks

Group 4 -

24114005	Aditya Bhadoria	Q1, Q2
24114006	Aditya Ranjan	Q3, Q4
24114028	Daddi Om	Q1, Q3
24114032	Deokar Parth	Q2, Q4
24114050	Kothawade Manthan	Q1, Q4
24114088	Satyam Sharma	Q2, Q3

Que 1: Acquiring the First Chunk in a BitTorrent Network

Mechanism for New Peers:

When a new peer like Alice joins a BitTorrent network (often referred to as a "leecher"), she initially has no chunks to share. She begins by contacting the "tracker" to obtain a list of active peers in the swarm. Because BitTorrent relies on a "tit-for-tat" mechanism (where peers upload to those who upload to them), Alice faces a bootstrapping problem: she cannot upload anything to encourage others to upload to her.

To solve this, BitTorrent uses a mechanism called optimistic unchoking. Existing peers periodically and randomly select a peer to "unchoke" (send data to) regardless of whether that peer has sent them data recently. Through this randomized altruism, Alice will eventually be optimistically unchoked by another peer, allowing her to acquire her very first chunk. Once she has a chunk, she can begin participating in the standard tit-for-tat exchange.

Fairness and Efficiency:

- **Fairness:** Optimistic unchoking is inherently fair because it prevents new or low-bandwidth users from being permanently locked out of the network. It gives every peer a chance to join the economy of the swarm.
- **Efficiency:** This mechanism continuously explores the network for better connections. A peer that Alice is optimistically unchoked by might discover that Alice has a very fast download/upload capacity, leading them to prioritize each other in the future.

Impact of Different Scenarios:

- **Network Congestion:** If the network is congested, the transfer rate of the optimistically unchoked chunk will be slow, extending the time Alice remains a pure leecher and delaying her active participation.
- **Peer Availability (Swarm Health):** In a healthy swarm with many "seeders" (peers who have the entire file and are only uploading), Alice will receive her first chunk very rapidly. In a swarm with very few peers, it may take several optimistic unchoking cycles (which occur every 30 seconds or so) before she is randomly selected.

Que 2: POP3 Challenges and Alternatives

Challenges of POP3 with "Download and Keep":

The Post Office Protocol version 3 (POP3) was originally designed for an era when users accessed their email from a single machine. While configuring POP3 to "download and keep a copy on the server" prevents emails from being deleted after downloading, it introduces severe synchronization limitations across multiple devices (e.g., a laptop, smartphone, and tablet):

- **No State Synchronization:** If a user reads an email on their phone, it will still appear as "unread" on their laptop.
- **Scattered Sent Items:** Emails sent from the phone are stored only on the phone's local storage and will not appear in the sent folder of the laptop.
- **Folder Management:** If a user organizes emails into folders on one device, those folders and movements are not reflected on the server or other devices.

Proposed Solutions/Alternatives:

- **IMAP (Internet Message Access Protocol):** This is the ideal alternative. IMAP is designed specifically to keep the state synchronized across multiple clients and the server. When an email is marked as read, moved to a folder, or deleted on one device, the IMAP client updates the server, and all other devices reflect the change.
- **Webmail (HTTP-based Access):** Users can bypass local email clients entirely and use a Web browser. Because the interface directly queries the centralized database on the server, the state is inherently unified regardless of which device is used to log in.

Que 3: SSL in the TCP/IP Stack

The Role of SSL: Secure Sockets Layer (SSL) provides critical process-to-process security services, which include encryption, data integrity, and end-point authentication. Importantly, SSL is not a third Internet transport protocol on the same level as TCP or UDP. Instead, it is an enhancement of TCP that is implemented at the application layer.

Steps for Application Developers:

- Integration: If a developer wants to utilize SSL services, they must explicitly include SSL code—typically existing, highly optimized libraries and classes—in both the client and server sides of their application.
- Socket API: The developer uses a specific SSL socket API, which operates similarly to the traditional TCP socket API.
- Data Flow: The sending process passes cleartext (unencrypted) data into the SSL socket. The SSL code within the sending host encrypts this data and then passes the encrypted payload down to the standard TCP socket for transmission over the network. At the destination, the receiving TCP socket passes the encrypted data up to SSL, which decrypts it and delivers cleartext to the receiving process.

Trade-offs:

Implementing SSL introduces computational overhead due to the encryption and decryption processes. It also introduces additional latency during the initial connection phase, as the client and server must perform an "SSL handshake" to negotiate cryptographic keys before any application data can be sent.

Que 4: Non-Persistent HTTP Connections and TCP

The Termination Mechanism: In a non-persistent HTTP connection, each TCP connection transports exactly one request message and one response message. After the HTTP server sends the requested object, the server process tells TCP to close the connection. However, to ensure reliability, the TCP protocol does not actually terminate the connection until it knows for sure that the client has received the response message intact.

Performance and Reliability Impacts:

- Overhead: Because the connection does not persist for other objects, a brand-new TCP connection must be established and maintained for every single requested object on a Web page. This requires TCP buffers to be allocated and TCP variables to be kept in both the client and the server for every single object, placing a significant burden on Web servers handling simultaneous requests.

- Latency: Each object suffers a delivery delay of two Round-Trip Times (RTTs). The first RTT is used to establish the TCP connection via a three-way handshake, and the second RTT is used to request and receive the object.

Comparison with Persistent Connections: In contrast, persistent connections (the default in HTTP/1.1) leave the TCP connection open after sending a response. Subsequent requests and responses between the same client and server can be sent over that same connection, completely avoiding the initial one-RTT penalty for connection setup on subsequent objects. Furthermore, persistent connections allow for pipelining, where multiple requests can be made back-to-back without waiting for previous replies.